WPT Day 10 Evening

Fri 30 Oct 26 · TypeScript generics (`<T>`) to remove duplicated response interfaces, running TypeScript with `tsx` / `tsx watch` and compiling with `tsc` , the `dev` / `build` / `start` scripts, then the start of React.js: library vs framework, the drawbacks of multi-page sites, Single Page Applications, scaffolding with Vite, the project structure, the execution flow (`main.jsx` → `App.jsx`), and components (class vs function)

Table of Contents

1. The Problem: Duplicated Response Types
2. Generics: Write Once, Pass the Type
3. Running TypeScript: tsx, Watch Mode and tsc
4. package.json Scripts: dev, build, start
5. React.js: Library vs Framework
6. Drawbacks of Multi-Page Applications
7. Single Page Applications
8. Setting Up React with Vite
9. React Project Structure
10. React App Execution Flow
11. Components: Definition and Composition
12. Class Components vs Function Components
13. Combined Worked Example
14. Quick Revision Checklist
15. Practice Exercises
16. Practice Answers

The Problem: Duplicated Response Types

Why this concept exists

A backend application deals with many **entities** — product, customer, order, cart. Every API reply to the client has the same outer envelope (did it work? what time? what is the payload?) but a different payload. With TypeScript we want each reply to be **typed**, so a naive approach is to write one response interface per entity.

The naive approach

```ts
// Separate entity interfaces
interface IProduct {
  id: number;
  name: string;
  price: number;
  quantity: number;
}
```

```ts
interface ICustomer {
  id: number;
  name: string;
  phone: string;
  email: string;
}

// Duplicated response interfaces — one per entity
interface IProductResponse {
  status: number;
  isSuccess: boolean;
  timestamp: Date;
  data: IProduct;
}

interface ICustomerResponse {
  status: number;
  isSuccess: boolean;
  timestamp: Date;
  data: ICustomer;
}

// Usage
let productResponse: IProductResponse = {
  status: 200,
  isSuccess: true,
  timestamp: new Date(),
  data: {
    id: 1,
    name: "Laptop",
    price: 999,
    quantity: 86
  }
};

let customerResponse: ICustomerResponse = {
  status: 200,
  isSuccess: true,
  timestamp: new Date(),
  data: {
    id: 1,
    name: "Aman",
    phone: "9876543210",
    email: "aman@mail.com"
  }
};
```

What is wrong with it

- The shape of every response is **identical**: `status`, `isSuccess`, `timestamp`, `data`.

- **Only the type of `data` differs** per entity.
- With 50 entities that means **50 separate response interfaces** — a clear **DRY** (Don't Repeat Yourself) violation.
- Maintenance nightmare: changing the response shape (say, adding a `message` field) means editing every interface.

```
IProductResponse   = { status, isSuccess, timestamp, data: IProduct  }
ICustomerResponse  = { status, isSuccess, timestamp, data: ICustomer }
IOrderResponse     = { status, isSuccess, timestamp, data: IOrder    }
                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^        ^^^^^^^^
                       always the same                         the only part that changes
```

The lecture's analogy: C / C++ / Java function duplication

The same problem exists in statically typed languages when writing arithmetic functions:

```c
// Three separate functions for the same addition operation
int intSum(int a, int b) { return a + b; }
float floatSum(float a, float b) { return a + b; }
double doubleSum(double a, double b) { return a + b; }
```

- The **operation** (addition) is identical; only the **data type** changes.
- Generics solve this by letting you write the logic once and **pass the type as an argument**.

> **Exam:** Duplication where the *operation/shape is the same but only the data type varies* is the exact symptom that **generics** are designed to remove.

Generics: Write Once, Pass the Type

The mechanism

A generic definition declares a **type parameter** inside angle brackets. That parameter is a *placeholder* that is filled in when the definition is *used*. It is like a normal function parameter — but the thing being passed is a **type**, not a value.

```typescript
// Single generic response interface
interface IEntityResponse<T> {
  status: number;
  isSuccess: boolean;
  timestamp: Date;
```

```
  data: T;   // T is a placeholder — resolved at usage time
}
```

```
// Usage: pass the specific entity type in angle brackets
let productResponse: IEntityResponse<IProduct> = {
  status: 200,
  isSuccess: true,
  timestamp: new Date(),
  data: {
    id: 1,
    name: "Laptop",
    price: 999,
    quantity: 86
  }
};

let customerResponse: IEntityResponse<ICustomer> = {
  status: 200,
  isSuccess: true,
  timestamp: new Date(),
  data: {
    id: 1,
    name: "Aman",
    phone: "9876543210",
    email: "aman@mail.com"
  }
};
```

How `T` is resolved

```
Declaration time                          Usage time
──────────────────────────────            ──────────────────────────────────────
interface IEntityResponse<T> {            IEntityResponse<IProduct>
  ...                                       T  =  IProduct    ⟶  data: IProduct
  data: T;
}                                         IEntityResponse<ICustomer>
                                            T  =  ICustomer   ⟶  data: ICustomer
```

- `T` is called a **type parameter** — a placeholder, **not a real type**.
- At **declaration time**, `T` is unknown; at **usage time**, you specify what `T` actually is.
- For `productResponse`, `T = IProduct`; for `customerResponse`, `T = ICustomer`.
- TypeScript still **enforces type safety**: if a required field is missing from `data`, an error is thrown at compile time.
- One interface now serves **all 50 entities** — no duplication, easy to maintain.

Mapping the lecture's analogy

Idea	Typed languages (functions)	TypeScript (interfaces)
Fixed part	the `a + b` logic	`status`, `isSuccess`, `timestamp`
Varying part	`int` / `float` / `double`	`IProduct` / `ICustomer` / …
Without generics	one function per type	one interface per entity
With generics	one definition, type passed in	`IEntityResponse<T>`

Background: the letter `T` stands for "Type" and is only a convention. Any name works (`IEntityResponse<TData>` is equally valid); `T` is used because it is short and universally recognised. The same angle-bracket syntax is seen in built-in types you will meet again, such as `Array<number>` (identical to `number[]`) and `Promise<string>`.

> **Exam:** *Rule from the lecture — when the operation is the same but the data type varies, use generics.* Syntax: declare with `interface Name<T> { data: T }`, use with `Name<IProduct>`.

> **Pitfall:** Writing `IEntityResponse` **without** the `<...>` is an error — `T` is unresolved, so TypeScript does not know the type of `data`. You must always supply the type argument when you use a generic interface.

Running TypeScript: tsx, Watch Mode and tsc

Why a special runner is needed

Node.js cannot run `.ts` files directly. There are two ways around it: *execute* the TypeScript through a helper during development, or *compile* it to JavaScript for deployment.

Normal mode with `tsx`

During development, use `tsx` (installed as a **dev dependency**) to execute `.ts` files directly without a manual compile step:

```
# Run a TypeScript file directly (one-shot execution)
npx tsx src/index.ts
```

- `npx` is used because `tsx` lives inside `node_modules`, **not in the global PATH**.

- The file runs **once** and the process exits.

Watch mode

Stopping and restarting the server after every change is tedious. `tsx` has a **watch mode** that monitors the file and **automatically restarts** the app on every save:

```
# Run with watch mode — auto-restarts on file save
npx tsx watch src/index.ts
```

- The process **stays alive** continuously.
- Any save to `index.ts` triggers an automatic restart — this is **hot reloading**.
- No need to press `Ctrl+C` and re-run manually.
- It is the equivalent of `nodemon` for plain JavaScript projects.

Compiling for production with `tsc`

For deployment (staging, production) TypeScript must be **compiled to JavaScript first**. The TypeScript compiler (`tsc`) reads `tsconfig.json` and converts every `.ts` file in `src/` into `.js` files inside `dist/` :

```
# Compile the entire TypeScript project to JavaScript
npx tsc
```

- Reads `tsconfig.json` for include/exclude rules and compiler options.
- Outputs compiled JS into the `dist/` folder.
- The generated JS has **no type annotations** — pure JavaScript that Node can run.
- **Never edit files inside** `dist/` — it is a build artifact only (it is overwritten on the next build).

```
DEVELOPMENT                          PRODUCTION
─────────────────────────            ──────────────────────────────────
src/index.ts                         src/index.ts
    |                                    |  npx tsc  (reads tsconfig.json)
    |  npx tsx watch src/index.ts        v
    v                                dist/index.js   (no types, plain JS)
runs directly, restarts on save          |  node dist/index.js
(no dist/ needed)                        v
                                     runs on the server
```

Comparison

	`npx tsx src/index.ts`	`npx tsx watch src/index.ts`	`npx tsc`
Purpose	Run once in development	Run continuously in development	Build for deployment
Output	Nothing written to disk	Nothing written to disk	`.js` files in `dist/`
Restarts on save?	No	**Yes** (hot reload)	No
JS equivalent	`node file.js`	`nodemon file.js`	—
Used by	Developer	Developer	DevOps / CI-CD

> **Exam:** `tsx` = run TS directly (dev). `tsx watch` = auto-restart on save (like `nodemon`). `tsc` = compile `src/*.ts` to `dist/*.js` using `tsconfig.json` (production). Never edit `dist/`.

---

package.json Scripts: dev, build, start

The problem

Typing long commands (`npx tsx watch src/index.ts`) repeatedly is error-prone and impractical for a team — each person remembers a different variation. The **`scripts`** block in `package.json` gives every command a short, memorable alias.

The mechanism

```
{
  "scripts": {
    "dev":   "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js"
  }
}
```

Background: npm adds `node_modules/.bin` to the PATH while it runs a script. That is why the scripts write `tsx` and `tsc` instead of `npx tsx` / `npx tsc` .

`npm run dev`

- Runs the app **locally in watch mode**.
- Used by **every developer** on their local machine during active development.
- Hot-reloads on every file save.

```
npm run dev
```

`npm run build`

- Compiles the entire TypeScript project to JavaScript via `tsc`.
- Used by **DevOps engineers or CI/CD pipelines** before deployment.
- **Abstracts away which compiler is being used** — the DevOps engineer just runs `npm run build`.

```
npm run build   # Creates dist/ folder with compiled JS files
```

`npm start` vs `npm run start`

- Launches the **compiled** application in production using Node directly.
- `start` is a **predefined key** in npm, so it does not need the `run` keyword.

```
# Both are equivalent — start is a reserved npm script name
npm start
npm run start

# Under the hood, this runs:
node dist/index.js
```

- `dev` and `build` are **custom keys** and always require `npm run`.
- `start` is special and works with just `npm start`.
- In production Node runs the compiled `.js` file — `tsx` **is never used in production**.

Summary table

Script	Command	Who runs it	When	Needs `run` ?
`dev`	`tsx watch src/index.ts`	Developer	Local development	**Yes** (`npm run dev`)
`build`	`tsc`	DevOps / CI-CD	Before deployment	**Yes** (`npm run build`)

Script	Command	Who runs it	When	Needs `run` ?
`start`	`node dist/index.js`	Server / production	After build	**No** (`npm start`)

```
Developer laptop:   npm run dev   ⟶  tsx watch  ⟶  hot reload
Deployment server:  npm run build ⟶  tsc        ⟶  dist/
                    npm start     ⟶  node dist/index.js
```

Exam: The order on a server is always **build first, then start** — `start` runs `dist/index.js`, which does not exist until `npm run build` has produced it.

Pitfall: `npm dev` fails (npm does not know it); it must be `npm run dev`. Only `start` (and a few other reserved names) can omit `run`.

React.js: Library vs Framework

What React is

React.js is a **JavaScript library** for building the **frontend (UI)** of web applications — specifically **Single Page Applications (SPAs)**. It is **not a framework**.

Library vs framework

	Library (React)	Framework (Angular)
Scope	Does **one thing well** — building UI components	Includes **everything** out of the box
Routing, HTTP, forms, state	Bring in third-party packages	Built in
Looking outside	Needed for other requirements	**Never needed** for frontend needs

- **Library:** for other needs (HTTP requests, routing, translations) you must bring in third-party packages.
- **Framework:** Angular is a full framework — routing, HTTP, forms and state management are included. You never need to look outside Angular for frontend needs.

- React was originally developed by **Facebook** to build their own application, then **open-sourced** for public use.

Why not just use HTML / CSS / JS?

You *can* build a complete frontend with plain HTML, CSS and JavaScript — and even call APIs using `fetch`. But from scratch this introduces serious problems:

- Every feature must be built manually: validation, DOM manipulation, routing, state.
- **Scalability** suffers as the codebase grows.
- Development is slow and effort-intensive.
- React provides these capabilities **out of the box**, dramatically simplifying frontend development.

> **Exam:** React = **library** (UI only, by Facebook, for SPAs). Angular = **framework** (everything included).

Drawbacks of Multi-Page Applications

The traditional approach

In a traditional (non-SPA) website — for example, a **school website** — the frontend is broken into multiple HTML files:

```
home.html
about.html
admissions.html
gallery.html
contact.html
```

Each page is a separate file, and navigating between them causes a **full browser reload**.

Drawback 1: No HTML code reusability

Every page needs the same navbar and footer. In plain HTML there is **no built-in mechanism** to reuse HTML across files:

- Navbar: 30 lines of HTML — must be copy-pasted into every `.html` file.
- Footer: 50 lines of HTML — same copy-paste problem.
- JavaScript and CSS **can** be reused via external files, but HTML cannot be modularised without significant custom DOM-manipulation effort.

Drawback 2: Hard to maintain and scale

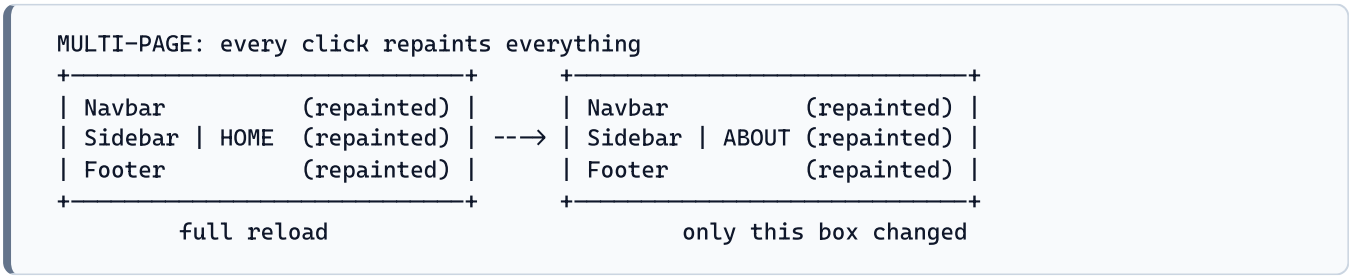
- A design change to the navbar requires editing **every single HTML file**.
- With 50 pages, that is **50 manual edits** for one change.
- Maintainability and scalability both suffer severely.

Drawback 3: Browser reloads on every page visit

This is the **most significant performance drawback**. Consider a school website where the navbar, sidebar and footer are identical on every page:

- User visits `home.html` → browser paints navbar, sidebar, footer **and** content (5–10 seconds for a heavy layout).
- User clicks "About Us" → browser **fully reloads**, repaints navbar, sidebar, footer and the new content.
- User clicks "Gallery" → browser reloads again, repaints everything from scratch.

The browser wastes time and resources repainting elements that have **not changed**. Only the middle content area is different — yet the entire page is recreated every time. This is not an efficient approach.



Summary

#	Drawback	Consequence
1	No HTML reusability	Navbar/footer copy-pasted into every file
2	Hard to maintain / scale	One change = edit N files
3	Reload on every visit	Unchanged elements repainted; slow

Single Page Applications

What SPA means

A Single Page Application has **only one HTML file**. The entire frontend — regardless of how large the application is — renders on that single file. Navigation between sections **does not reload the browser**; instead, content is **swapped dynamically based on the URL**.

- **First load:** the full layout (navbar, sidebar, footer) is painted **once**.
- **Subsequent navigation:** only the **content area** updates; layout elements stay intact.
- The browser is **never reloaded** when clicking links.

```
SPA: layout painted once, only the content area is swapped
+-----------------------------------+      +-----------------------------------+
| Navbar           (untouched) |      | Navbar           (untouched) |
| Sidebar | HOME  <-- swapped | --> | Sidebar | ABOUT <-- swapped |
| Footer           (untouched) |      | Footer           (untouched) |
+-----------------------------------+      +-----------------------------------+
```

Real-world examples

Notion (demonstrated live during the session):

- Navigating between "New Chat", "Library" and individual pages — the URL changes, the content changes, but the **sidebar never reloads**.
- The browser does not reload; only the content area is replaced.

YouTube:

- Clicking a video link changes the URL (`/watch?v=...`) **without a page reload**.
- Clicking a recommended video again — no reload, content replaced in place.
- Searching for "JavaScript tutorial" and clicking results — same behaviour.

Other SPAs: Gmail, Google Drive, Facebook, Instagram, WhatsApp Web, Outlook — all operate as SPAs. Clicking Inbox, Sent, Spam or any section never triggers a browser reload.

How an SPA works internally

```
index.html  <-- the only HTML file; never changes
    |
    └── Based on URL --> renders different components dynamically

        /home      --> renders Home component
        /about     --> renders About component
        /contact   --> renders Contact component
```

- The HTML file stays the same; **JavaScript swaps the content area**.
- React handles this routing and conditional rendering out of the box.
- Result: a **software-like user experience** — it feels like a desktop or mobile app, not a traditional website.

Multi-page vs SPA

	Multi-page application	Single Page Application
HTML files	Many (`home.html` , `about.html` , ...)	**One** (`index.html`)
Navigation	Full browser reload	**No reload**; content swapped
Layout (navbar/footer)	Repainted every time	Painted **once**
Reuse of navbar/footer	Copy-paste	Reusable component
Feel	Website	Software / app
Examples	School website	Notion, YouTube, Gmail

> **Exam:** SPA = **one HTML file**, URL-driven dynamic content swapping, **no browser reload**, layout painted once.

Background: the lecture says React "handles routing out of the box"; strictly, URL-to-component routing in React is normally provided by an add-on package (React Router), consistent with the lecture's own point that React is a *library* that relies on third-party packages for routing. Treat routing as a topic still to come.

Setting Up React with Vite

What Vite is

Vite is a **CLI scaffolding tool** that generates a complete starter template for various frontend frameworks — React, Angular, Vue or plain JavaScript. It handles the boilerplate so you do not manually create `package.json` , install dependencies or configure the build system.

Creating a React app

```
# Scaffold a new React application
```

```
npx create-vite@latest
```

Vite then prompts interactively:

1. **Project name:** e.g. `my-react-app` (creates a folder with this name).
2. **Framework:** choose **React**.
3. **Variant: JavaScript** (or TypeScript — covered later).
4. **Linter:** ESLint (discussed separately).
5. **Install and start now?:** choose **No** to do it manually.

Installing dependencies and running

```
# Navigate into the project folder
cd my-react-app

# Install all dependencies
npm i

# Launch the development server
npm run dev
# App runs at http://localhost:5173
```

npx vs npm

	`npm`	`npx`
Purpose	**Package manager** — installs and manages dependencies	**Package runner** — executes packages directly
Requires installation first?	Yes	**No**
Disk usage	Installs to disk	Runs temporarily without installing

- `create-vite` was **never separately installed** — `npx` ran it directly from the registry.
- This saves disk space and avoids polluting global installations.
- `npx` has all of npm's capabilities plus the ability to run packages without prior installation.

The whole flow

```
npx create-vite@latest   ⟶ answer prompts (name, React, JavaScript, ...)
cd my-react-app
```

```
npm i                    ⟶ creates node_modules/
npm run dev              ⟶ dev server at http://localhost:5173
```

> **Exam:** `npx create-vite@latest` scaffolds; `npm i` installs; `npm run dev` serves at `localhost:5173`. `npx` runs a package **without installing it**.

React Project Structure

After scaffolding and installing dependencies the project looks like this:

```
my-react-app/
├── index.html          ← The single HTML file for the entire SPA
├── vite.config.js      ← Vite tool configuration and behavior
├── package.json        ← Dependencies and scripts (dev, build, preview, lint)
├── package-lock.json   ← Auto-generated lockfile
├── README.md           ← Project description
├── public/             ← Static assets (images, icons) served as-is
├── node_modules/       ← Installed dependencies (after npm i)
└── src/                ← All your source code lives here
    ├── main.jsx        ← Entry point – mounts App component
    ├── App.jsx         ← Root App component (first component rendered)
    └── assets/         ← Images and other assets used in components
```

Key points

- `index.html` is the **only HTML file** — the entire React app renders inside it; **never edit it directly**.
- `src/` is where **all components** are created and managed.
- `public/` is for **static files** that need to be served directly.
- `vite.config.js` allows further customisation of the Vite build tool's behaviour.
- The pre-configured `package.json` scripts:

```
{
  "scripts": {
    "dev":     "vite",
    "build":   "vite build",
    "lint":    "eslint .",
    "preview": "vite preview"
  }
}
```

File / folder	Role
`index.html`	The one HTML page of the SPA
`vite.config.js`	Configure Vite
`package.json`	Dependencies + scripts
`package-lock.json`	Auto-generated lockfile
`public/`	Static assets served as-is
`node_modules/`	Installed dependencies
`src/main.jsx`	Entry point
`src/App.jsx`	Root component
`src/assets/`	Images used inside components

Background: the scripts here are the same idea as the TypeScript scripts earlier today — `dev` is for developers, `build` produces the deployable output (`vite build` writes a `dist/` folder), and `preview` serves that built output locally for a quick check. `lint` runs ESLint over the code to report style/bug problems.

React App Execution Flow

How the app starts

When you run `npm run dev` and open `localhost:5173` , React follows a specific order:

```
npm run dev
    │
    └── Vite starts the dev server
            │
            └── index.html is served
                    │
                    └── main.jsx is loaded first   ←── entry point
                            │
                            └── App component is mounted and rendered
                                    │
                                    └── App's JSX is displayed in the browser
```

`main.jsx` — the entry point

`main.jsx` is the starting point of every React application. It is responsible for **mounting the `App` component onto the single HTML file**:

- Control **always enters `main.jsx` first**.
- `main.jsx` tells React to render the `App` component inside `index.html`.
- This is why the `App` component is always the first thing displayed.

Background: in a Vite-generated project `main.jsx` looks like the code below (the lecture describes its role but does not show the code, so this is the standard template). `index.html` contains an empty `<div id="root"></div>`, and `createRoot( ... ).render( ... )` puts the `App` component inside it.

```jsx
// src/main.jsx (standard Vite template)
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

`App.jsx` — the root component

The `App` component is the **root of the entire component tree**. It is the first component rendered when the application launches. After cleaning up the Vite-generated boilerplate:

```jsx
// App.jsx — cleaned up to show custom content
function App() {
  return (
    <div>
      <h1>Hello World</h1>
    </div>
  );
}

export default App;
```

- Whatever is inside `App.jsx` appears **immediately** in the browser.
- **Hot reloading:** saving any change to `App.jsx` instantly updates the browser — no manual refresh needed.

- All other components are eventually rendered **through or within** the `App` component.

```
index.html  (<div id="root">)
   └── main.jsx  mounts ──→  App
                              ├── Navbar
                              ├── (page content components)
                              └── Footer
```

> **Exam:** Execution order: `npm run dev` → Vite dev server → `index.html` → `main.jsx` **first** → `App` component → JSX shown in the browser. `App` is the **root** component.

Components: Definition and Composition

What is a component?

A component is any **UI element visible on the screen**. This definition is intentionally broad:

- **Atomic-level elements:** a button, an input field, a heading (`<h1>`), an image, a link, a table, a video player.
- **Composite elements:** a navbar (containing multiple links), a sidebar, a footer.
- **Full screens:** a Login screen, a Dashboard screen, a Registration form — each is itself a component.

```
Login Screen (component)
├── Navbar (component)
│   ├── Logo (component)
│   └── Links (component)
├── Form (component)
│   ├── Input: Email (component)
│   ├── Input: Password (component)
│   └── Button: Submit (component)
└── Footer (component)
```

- A component can contain other components — this is called **component composition**.
- In React, instead of creating multiple HTML pages, you create **multiple components**.
- Each component lives in its **own** `.jsx` **or** `.js` **file**.

SPA design principle

```
Traditional (Multi-Page):          React SPA:

home.html                          HomeComponent.jsx
about.html          ──→            AboutComponent.jsx
```

```
contact.html                    ContactComponent.jsx
admissions.html                 AdmissionsComponent.jsx
```

Instead of creating multiple pages, you create multiple components. React renders the appropriate component based on the current URL — all within the single `index.html`.

How components fix the multi-page drawbacks

Multi-page drawback	How components solve it
Navbar/footer copy-pasted in every file	Write `Navbar` / `Footer` **once**, use it anywhere
50 pages = 50 edits	Edit one component file
Full reload each visit	Only the changing component is swapped

> **Exam:** *Component = any UI element visible on screen* (from a button to a whole screen). Components nest — **component composition**. React replaces "many HTML pages" with "many components".

Class Components vs Function Components

Two types of components

React provides two syntaxes for creating components:

	Class component	Function component
Syntax	ES6 `class`	JavaScript `function`
Available since	React's beginning	Later React updates
OOP support	Yes (inheritance, etc.)	No
Weight	Heavier	**Lightweight**
Performance	Slower	**Better**
Modern usage	Rare (specific cases)	**Majority** of modern apps

Class components

A class component uses ES6 class syntax. Simply creating a class is **not enough** — it must **extend React's built-in `Component` class** to gain the properties and behaviours required to function as a React component:

```jsx
// Navbar.jsx – Class Component
import { Component } from 'react';

class Navbar extends Component {
  render() {
    return (
      <div>
        <a href="/home">Home</a>
        <a href="/about">About</a>
        <a href="/contact">Contact</a>
      </div>
    );
  }
}

export default Navbar;
```

```jsx
// Footer.jsx – Class Component
import { Component } from 'react';

class Footer extends Component {
  render() {
    return (
      <div>
        <p>© 2026 My App. All rights reserved.</p>
      </div>
    );
  }
}

export default Footer;
```

- `import { Component } from 'react'` — imports the **base `Component` class** from the React package.
- `class Navbar extends Component` — `Navbar` **inherits** all React component properties via this inheritance.
- **Without `extends Component`**, the class is just a plain JavaScript class with no React capabilities.
- The same pattern applies to every class component: `class <Name> extends Component`.

- OOP principles (inheritance, method overriding) are available inside class components — `render()` is the method that returns what to draw.

Function components

A function component is simply a **JavaScript function that returns JSX**. It is lighter, simpler and performs better:

```jsx
// Navbar.jsx – Function Component (modern approach)
function Navbar() {
  return (
    <div>
      <a href="/home">Home</a>
      <a href="/about">About</a>
      <a href="/contact">Contact</a>
    </div>
  );
}

export default Navbar;
```

- No class syntax, no inheritance, no `extends`.
- The `App` component generated by Vite is itself a **function component** (as seen in `App.jsx`).
- Function components are **lightweight** — less overhead than class components.
- Modern React applications use function components almost exclusively.

Side by side

```
CLASS                                   FUNCTION
import { Component } from 'react';      (no import needed for the component itself)

class Navbar extends Component {        function Navbar() {
  render() {                              return (
    return ( <div> … </div> );              <div> … </div>
  }                                       );
}                                       }
export default Navbar;                  export default Navbar;
```

Why start with class components?

Fundamental React concepts — **state** and **lifecycle methods** — are native to class components. They were built into the class syntax from the beginning. Function components gained equivalent capabilities **later through a different mechanism (hooks)**. Understanding the class-based version first makes it easier to understand what hooks are solving when function components are introduced.

- Learn **state** and **lifecycle** in class components first.
- Then implement the same concepts in function components.
- From that point forward, use **function components exclusively** for all new development.

Background: in the examples the links use plain `<a href>`. A plain anchor still triggers a full browser reload, which is exactly what an SPA tries to avoid. The lecture used them only to show component syntax; SPA-style navigation without reload comes with React's routing, which is a later topic.

> **Exam:** Class component = `class X extends Component { render() { return JSX } }`. Function component = `function X() { return JSX }`. Both end with `export default X`. Function components are lighter and modern; class components are taught first because **state and lifecycle** were native to them (function components later got **hooks**).

> **Pitfall:** A class without `extends Component` is **not** a React component — it is just a JavaScript class.

Combined Worked Example

This ties the day together: a **generic typed response** with scripts for a TypeScript backend, then a **React app** using a class component and a function component composed inside `App`.

Part A — TypeScript backend with a generic response

`package.json`:

```
{
  "name": "shop-api",
  "version": "1.0.0",
  "scripts": {
    "dev":   "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js"
  }
}
```

`src/index.ts`:

```
// ——— Entity shapes ———
interface IProduct {
```

```
  id: number;
  name: string;
  price: number;
  quantity: number;
}

interface ICustomer {
  id: number;
  name: string;
  phone: string;
  email: string;
}

// ——— ONE generic response envelope for every entity ———
interface IEntityResponse<T> {
  status: number;
  isSuccess: boolean;
  timestamp: Date;
  data: T;          // T is resolved at usage time
}

// ——— Usage 1: T = IProduct ———
const productResponse: IEntityResponse<IProduct> = {
  status: 200,
  isSuccess: true,
  timestamp: new Date(),
  data: { id: 1, name: "Laptop", price: 999, quantity: 86 }
};

// ——— Usage 2: T = ICustomer ———
const customerResponse: IEntityResponse<ICustomer> = {
  status: 200,
  isSuccess: true,
  timestamp: new Date(),
  data: { id: 1, name: "Aman", phone: "9876543210", email: "aman@mail.com" }
};

console.log(productResponse.data.name);    // Laptop
console.log(customerResponse.data.email);  // aman@mail.com
```

Commands:

```
npm run dev       # developer: tsx watch, restarts on every save
npm run build     # DevOps: tsc → dist/index.js
npm start         # server: node dist/index.js
```

Part B — React app (Vite, JavaScript variant)

```
npx create-vite@latest        # name: my-react-app, React, JavaScript, No to "install now"
cd my-react-app
npm i
npm run dev                   # http://localhost:5173
```

`src/Navbar.jsx` (class component):

```
import { Component } from 'react';

class Navbar extends Component {
  render() {
    return (
      <div>
        <a href="/home">Home</a>{' '}
        <a href="/about">About</a>{' '}
        <a href="/contact">Contact</a>
      </div>
    );
  }
}

export default Navbar;
```

`src/Footer.jsx` (function component):

```
function Footer() {
  return (
    <div>
      <p>© 2026 My App. All rights reserved.</p>
    </div>
  );
}

export default Footer;
```

`src/App.jsx` (root component — composes the others):

```
import Navbar from './Navbar.jsx';
import Footer from './Footer.jsx';

function App() {
  return (
    <div>
      <Navbar />
```

```
      <h1>Hello World</h1>
      <Footer />
    </div>
  );
}

export default App;
```

`src/main.jsx` stays as generated by Vite (it mounts `<App />` into `index.html`).

End-to-end trace

1. **Type check (Part A):** `IEntityResponse<IProduct>` makes `T = IProduct`, so `data` must have `id`, `name`, `price`, `quantity`. Leaving out `price` would be a compile error. `IEntityResponse<ICustomer>` makes `data` an `ICustomer` instead — from the same single interface.
2. **Development:** `npm run dev` runs `tsx watch src/index.ts`. Saving the file restarts it, printing `Laptop` and `aman@mail.com`.
3. **Deployment:** the pipeline runs `npm run build` → `tsc` reads `tsconfig.json`, writes `dist/index.js` with the types stripped. The server runs `npm start` → `node dist/index.js`. `tsx` is not involved.
4. **React startup (Part B):** `npm run dev` starts Vite's dev server; the browser opens `localhost:5173` and receives `index.html` (the only HTML file).
5. **Entry point:** `index.html` loads `main.jsx`, which mounts `App` in the page.
6. **Composition:** `App` returns a `div` containing `<Navbar />` (class component, runs its `render()`), the `<h1>`, and `<Footer />` (function component). Navbar and Footer are each written once and could be reused in any other component.
7. **Hot reload:** editing `Footer.jsx` and saving updates the browser immediately, with no manual refresh.

Quick Revision Checklist

Topic	One-line takeaway	Topic	One-line takeaway
Duplicate interfaces	Same shape, different `data` type per entity = DRY violation	Generics	Same operation, varying type → use `<T>`

Topic	One-line takeaway	Topic	One-line takeaway
Type parameter `T`	Placeholder, unknown at declaration, resolved at usage	`IEntityResponse<IProduct>`	Passing the type in angle brackets sets `T = IProduct`
`npx tsx file.ts`	Runs TS directly, once, no compile step	`npx tsx watch file.ts`	Restarts on every save (like `nodemon`)
`npx tsc`	Compiles `src/*.ts` → `dist/*.js` via `tsconfig.json`	`dist/`	Build artifact; never edit it
`npm run dev`	`tsx watch src/index.ts` — local development	`npm run build`	`tsc` — used by DevOps/CI-CD before deploy
`npm start`	`node dist/index.js` — production; `run` optional	`tsx` in production	Never — Node runs compiled JS
React	JavaScript **library** for SPA frontends, by Facebook	Angular	Full **framework**, everything built in
Multi-page drawbacks	No HTML reuse, hard to maintain, reload on every visit	SPA	One HTML file; content swapped by URL; no reload
SPA examples	Notion, YouTube, Gmail, Drive, Facebook, Instagram, WhatsApp Web	Vite	CLI scaffolding tool that generates the starter template
`npx create-vite@latest`	Scaffold; `npm i` then `npm run dev` → `localhost:5173`	npx vs npm	npx runs a package without installing; npm installs
`index.html`	The single HTML file; never edit directly	`src/`	All components live here
`main.jsx`	Entry point; always entered first; mounts `App`	`App.jsx`	Root component; first one rendered
Component	Any UI element visible on screen, from button to whole screen	Composition	Components can contain other components
Class component	`class X extends Component`, heavier, slower,	Function component	Plain function returning JSX; light,

Topic	One-line takeaway	Topic	One-line takeaway
	OOP		modern
Why class first	State and lifecycle were native to classes	Hooks	How function components later got the same powers

---

Practice Exercises

Concept

1. State the problem that generic interfaces solve, using the product/customer response example.
2. What is a type parameter? Why is `T` "not a real type"?
3. Explain the C/C++/Java analogy used to introduce generics.
4. Why do we use `npx tsx` rather than `tsx` directly in the terminal?
5. Difference between `npx tsx src/index.ts` and `npx tsx watch src/index.ts`? Which JS tool is equivalent to watch mode?
6. Who uses `npm run dev`, and who uses `npm run build`? Why is it useful that the DevOps engineer just runs `npm run build`?
7. Why does `npm start` work without `run`, while `npm run dev` needs it?
8. Why is React called a library, and how does that differ from Angular?
9. List the three drawbacks of multi-page applications.
10. What is an SPA, and how does navigation work inside it?
11. Compare `npm` and `npx`. Why was `create-vite` never "installed"?
12. What is the role of `index.html`, `main.jsx` and `App.jsx` in a Vite React project?
13. What is a component? Give one atomic, one composite and one full-screen example.
14. Give four differences between class components and function components. Why does the course still start with class components?

Application

15. You have `IOrder { id: number; total: number }`. Using `IEntityResponse<T>`, declare a variable `orderResponse` for a successful (status 200) order with id 7 and total 1500.
16. The team wants every response to also carry a `message: string`. How many places must change with generics, versus the 50-interface approach?

17. Write the three `package.json` scripts for a TypeScript project whose entry file is `src/index.ts` and compiled file is `dist/index.js`.
18. Write the commands to create a React app called `school-app` with Vite, install it and run it. At what address does it run?
19. Write a class component `Header` and a function component `Header` that each render `<h1>School</h1>`.

Predict the output / behaviour

20. After running `npm run build`, which folder appears, and does its content contain type annotations?
21. A developer saves `index.ts` while `npm run dev` is running. What happens? What happens if they instead started the app with `npx tsx src/index.ts`?
22. On a fresh server, someone runs `npm start` before ever running `npm run build`. What happens and why?
23. In a multi-page school website with 50 pages, the navbar colour must change. How many files are edited? How many in a React SPA built with a `Navbar` component?

Spot the bug

24. `let r: IEntityResponse = { status: 200, isSuccess: true, timestamp: new Date(), data: { id: 1, name: "Laptop", price: 999, quantity: 86 } };` — what is wrong?
25. `let r: IEntityResponse<IProduct> = { status: 200, isSuccess: true, timestamp: new Date(), data: { id: 1, name: "Laptop", price: 999 } };` — what is wrong?
26. Someone fixes a bug directly inside `dist/index.js`. What is wrong with that?
27. `npm dev` is typed to start the development server. What is wrong?
28. `class Navbar { render() { return <div>Menu</div>; } }` is used as a React component. Why will it not behave as one?
29. A developer opens `index.html` in a Vite React project and starts adding `<h1>` tags and a copy of the navbar for each page. What principle of SPAs/React is being broken?

Practice Answers

1. Every entity (product, customer, ...) needed its own response interface, although `status`, `isSuccess` and `timestamp` are identical and only `data` differs. With 50 entities that means 50 near-identical interfaces (a DRY violation) that all need editing when the response shape changes. A single `IEntityResponse<T>` removes the duplication.

2. A type parameter is a placeholder for a type written in angle brackets (`<T>`). It is not a real type because at declaration time nobody knows what it is; it becomes a real type (`IProduct` , `ICustomer` , ...) only at usage time when you supply it.

3. In C/C++/Java you would write `intSum` , `floatSum` , `doubleSum` — the same `a + b` logic repeated only because the data type changes. Generics let you write the logic once and pass the type as an argument. Likewise one `IEntityResponse<T>` replaces a response interface per entity.

4. `tsx` is installed locally inside `node_modules` (as a dev dependency), not in the global PATH, so `npx` is used to locate and run it.

5. `npx tsx src/index.ts` runs the file once and the process exits. `npx tsx watch src/index.ts` keeps the process alive and restarts the app automatically on every save (hot reloading). The equivalent JavaScript tool is `nodemon` .

6. Developers run `npm run dev` locally; DevOps engineers / CI-CD pipelines run `npm run build` before deployment. The script abstracts which compiler is used — they never need to know it is `tsc` .

7. `start` is a predefined (reserved) key in npm, so `npm start` is a shortcut for `npm run start` . `dev` and `build` are custom keys, so they always need `npm run` .

8. React does one thing — building UI components — and for HTTP requests, routing, translations etc. you add third-party packages: a library. Angular is a framework that includes routing, HTTP, forms and state management, so you never look outside it.

9. (1) No HTML code reusability — navbar/footer copy-pasted in every file. (2) Hard to maintain and scale — one design change means editing every file. (3) The browser reloads and repaints everything on every page visit, including unchanged elements.

10. A Single Page Application has exactly one HTML file. The layout (navbar, sidebar, footer) is painted once; navigating changes the URL and swaps only the content area dynamically — with no browser reload.

11. `npm` is a package manager that installs and manages dependencies on disk. `npx` is a package runner that executes a package directly without installing it first. `create-vite` was run temporarily by `npx` from the registry, so it was never installed (saving disk space and avoiding global pollution).

12. `index.html` is the single HTML file the entire app renders inside (never edited directly). `main.jsx` is the entry point — control always enters it first, and it mounts `App` onto the page. `App.jsx` is the root component — the first component rendered, through which all others are shown.

13. A component is any UI element visible on screen. Atomic: a button / input / heading. Composite: a navbar / sidebar / footer. Full screen: a Login screen / Dashboard.

14. Class: ES6 class, supports OOP, heavier, slower, available since the beginning, rare today. Function: JavaScript function, no OOP, lightweight, better performance, added in later updates, used by the majority of modern apps. The course starts with class components because state and lifecycle methods are native to them; function components got the same capabilities later via hooks, which are easier to understand after seeing the class version.

15. Declaration:

```ts
interface IOrder {
  id: number;
  total: number;
}

let orderResponse: IEntityResponse<IOrder> = {
  status: 200,
  isSuccess: true,
  timestamp: new Date(),
  data: { id: 7, total: 1500 }
};
```

16. With generics: **one** place (add `message: string` to `IEntityResponse<T>`). With separate interfaces: all 50 interfaces must be edited.

17.

```json
{
  "scripts": {
    "dev":   "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js"
  }
}
```

18.

```bash
npx create-vite@latest      # name: school-app, framework: React, variant: JavaScript, install now: No
cd school-app
npm i
npm run dev
```

It runs at `http://localhost:5173`.

19. Class version:

```
import { Component } from 'react';

class Header extends Component {
  render() {
    return <h1>School</h1>;
  }
}

export default Header;
```

Function version:

```
function Header() {
  return <h1>School</h1>;
}

export default Header;
```

20. A `dist/` folder appears containing the compiled `.js` files. They have **no type annotations** — plain JavaScript Node can run.

21. With `npm run dev` (watch mode) the app restarts automatically with the new code. With `npx tsx src/index.ts` (normal mode) nothing happens on save — the process ran once; it must be re-run manually.

22. It fails: `npm start` runs `node dist/index.js`, and `dist/` does not exist until `npm run build` has compiled the TypeScript. Build first, then start.

23. Multi-page site: all **50** files are edited. React SPA: **one** file — the `Navbar` component — because it is written once and reused.

24. `IEntityResponse` is generic but no type argument was given, so `T` is unresolved. It must be `IEntityResponse<IProduct>`.

25. The `data` object is missing the required `quantity` field of `IProduct`. TypeScript raises an error because the type `T = IProduct` is enforced.

26. `dist/` is a build artifact regenerated by `tsc`; the next `npm run build` overwrites the edit, so the fix is lost. Edit the `.ts` file in `src/` instead.

27. `dev` is a custom script key, so it requires `run`: `npm run dev`. Only `start` can omit `run`.

28. It does not `extend Component` (imported from `'react'`), so it is just a plain JavaScript class with none of the properties React needs for a component.

29. In an SPA/React app `index.html` is the single HTML file that must not be edited directly; pages should be separate components (e.g. `Navbar`, `Footer`) in `src/`, written once and rendered by React — copying markup per page brings back the multi-page drawbacks (no reuse, hard to maintain).